



Pattern Recognition
and Applications Lab

La Programmazione ad Oggetti in Python

Docente: Ambra Demontis

Anno Accademico: 2025 - 2026

Corso di Laurea in Ingegneria Elettronica, Informatica e delle
Telecomunicazioni



University of Cagliari,
Italy

Department of Electrical and
Electronic Engineering



La Programmazione ad Oggetti in Python

- Gli oggetti di tipo funzione.
- Numero variabile di argomenti.
- I decorator.

Gli Oggetti di Tipo Funzione

Le funzioni sono oggetti appartenenti alla classe built-in **function**.
Vengono creati automaticamente quando la funzione viene definita.

```
def incrementa_elementi(lista, valore_incremento = 1):  
    for elem_idx in range(len(lista)):  
        lista[elem_idx] = lista[elem_idx] + valore_incremento  
  
print(type(incrementa_elementi))
```

Stampa:

```
<class 'function'>
```

Gli Oggetti di Tipo Funzione

Anche le funzioni hanno attributi, come il loro nome e i valori di default della funzione.

```
print("nome funzione : ", incrementa_elementi.__name__)  
print("valori di default: ", incrementa_elementi.__defaults__)
```

Stampa:

nome funzione : incrementa_elementi

valori di default: (1,)

Gli Oggetti Invocabili (Callable Object)

E' possibile creare oggetti che è possibile invocare come se fossero funzioni.

Perchè questo sia possibile, la loro classe di appartenenza deve implementare il metodo speciale **`__call__`**.

Questo metodo definisce il comportamento che l'oggetto deve avere quando viene invocato.

Gli Oggetti Invocabili (Callable Object)

```
class CStampaMessaggioLogin:
```

```
    @staticmethod
```

```
    def __call__(username):
```

```
        print("Benvenuto utente {:}. Ti ricordiamo che dopo 30 minuti "  
              "di inattività dovrai ri-effettuare il login.".format(username))
```

```
stampam_messaggio_login = CStampaMessaggioLogin()
```

```
username = "anna_bianchi"
```

```
stampam_messaggio_login(username)
```

Stampa:

Benvenuto utente anna_bianchi. Ti ricordiamo che dopo 30 minuti di inattività dovrai ri-effettuare il login.

Gli Oggetti di Tipo Funzione

E' possibile passare un oggetto funzione come argomento a funzioni/metodi.

```
def eleva_elementi(lista, potenza= 2):  
    for elem_idx in range(len(lista)):  
        lista[elem_idx] = lista[elem_idx] ** potenza
```

Problema: log deve conoscere i
parametri della funzione che applica

```
def log(funzione, lista):  
    print("applica la funzione '", funzione.__name__, "' alla lista ", lista)  
    funzione(lista)
```

```
lista = [1, 2, 3]  
log(eleva_elementi, lista)  
print(lista)
```

Stampa:

```
applica la funzione ' eleva_elementi ' alla lista  [1, 2, 3]  
[1, 4, 9]
```

Numero Variabile di Argomenti

E' possibile creare funzioni che ricevono un numero variabile di argomenti passati senza la keyword (chiamati **vararg**)..

```
def eleva_elementi(lista, potenza=2):  
    for elem_idx in range(len(lista)):  
        lista[elem_idx] = lista[elem_idx] ** potenza
```

```
def log(funzione, *varargs):  
    print("applica la funzione '", funzione.__name__, "' passando come argomenti ai parametri senza  
keyword: ", varargs)  
    funzione(*varargs)
```

```
lista = [1, 2, 3]  
log(eleva_elementi, lista)  
print(lista)
```

```
applica la funzione ' eleva_elementi ' passando come argomenti ai parametri senza keyword:  ([1, 2,  
3],)  
[1, 4, 9]
```


Numero Variabile di Argomenti

..e con un numero variabile di argomenti per argomenti passati con la keyword (chiamati **kwarg**)

```
def eleva_elementi(lista, potenza=2):  
    for elem_idx in range(len(lista)):  
        lista[elem_idx] = lista[elem_idx] ** potenza  
  
def log(funzione, *varargs, **kwargs):  
    print("applica la funzione '", funzione.__name__, "' passando come argomenti ai  
    parametri senza keyword ", varargs, " e con valore di keyword: ", kwargs)  
    funzione(*varargs, **kwargs)  
  
lista = [1, 2, 3]  
log(eleva_elementi, lista, potenza=3)  
print(lista)
```

Numero Variabile di Argomenti

Verrà stampato:

```
applica la funzione ' eleva_elementi ' passando come argomenti ai parametri senza  
keyword: ([1, 2, 3],) e con keyword: {'potenza': 3}  
[1, 8, 27]
```

Numero Variabile di Argomenti

NB: la cosa importante è come vengono passati non come sono definiti.

```
def eleva_elementi(lista, potenza=2):  
    for elem_idx in range(len(lista)):  
        lista[elem_idx] = lista[elem_idx] ** potenza  
  
def log(funzione, *varargs, **kwargs):  
    print("applica la funzione '", funzione.__name__, "' passando come argomenti ai parametri  
senza keyword: ", varargs, " e con keyword: ", kwargs)  
    funzione(*varargs, **kwargs)  
  
lista = [1, 2, 3]  
log(eleva_elementi, lista, 3)  
print(lista)
```

Numero Variabile di Argomenti

Verrà stampato:

```
applica la funzione ' eleva_elementi ' passando come argomenti ai parametri senza  
keyword: ([1, 2, 3], 3) e con keyword: {}  
[1, 8, 27]
```

Decoratori

Generalmente si sfruttano i decoratori che ci permettono di andare a modificare il comportamento di una funzione.

```
def log(funzione):  
    def wrapper (*varargs, **kwargs): #inscatola la funzione ricevuta  
        print("applica la funzione '", funzione.__name__, "' passando come argomenti ai  
parametri keyword: ", varargs, " e con keyword: ", kwargs)  
        funzione(*varargs, **kwargs)  
    return wrapper  
  
def eleva_elementi(lista, potenza= 2):  
    for elem_idx in range(len(lista)):  
        lista[elem_idx] = lista[elem_idx] ** potenza  
  
lista = [1, 2, 3]  
eleva_elementi_wrappata = log(eleva_elementi)  
eleva_elementi_wrappata(lista, potenza=3)  
print(lista)
```

Numero Variabile di Argomenti

Verrà stampato:

```
applica la funzione ' eleva_elementi ' passando come argomenti ai parametri senza  
keyword: ([1, 2, 3],) e con keyword: {'potenza': 3}  
[1, 8, 27]
```

Decoratori

In Python esiste una sintassi ad-hoc.

```
def log(funzione):
    def wrapper (*varargs, **kwargs):
        print("applica la funzione '", funzione.__name__, "' passando come argomenti ai
parametri senza keyword: ", varargs, " e con keyword: ", kwargs)
        funzione(*varargs, **kwargs)
    return wrapper

@log
def eleva_elementi(lista, potenza=2):
    for elem_idx in range(len(lista)):
        lista[elem_idx] = lista[elem_idx] ** potenza

lista = [1, 2, 3]
eleva_elementi(lista, potenza=3)
print(lista)
```

Numero Variabile di Argomenti

Verrà stampato:

```
applica la funzione ' eleva_elementi ' passando come argomenti ai parametri senza  
keyword: ([1, 2, 3],) e con keyword: {'potenza': 3}  
[1, 8, 27]
```


Esercizio sui Decoratori

Creare un decoratore che simula il comportamento del decoratore classmethod quando viene invocato con la sintassi <referimento all'istanza>.nome_dell'attributo e verificarne il funzionamento.

Esercizio sui Decoratori

```
class CClassEsempio():  
  
    @myclassmethod  
    def stampa_nome_classe(cls, *vargs, **kwargs):  
        print("classe: ", cls)  
        print("vargs: ", *vargs)  
        print("kwargs ", kwargs)  
  
oggetto_esempio = CClassEsempio()  
oggetto_esempio.stampa_nome_classe( 2, 3, quattro=4, cinque=5)
```

classe: <class '__main__.CClassEsempio'>

vargs: 2 3

kwargs {'quattro': 4, 'cinque': 5}

Esercizio sui Decoratori

```
def myclassmethod(funzione):  
    def wrapper(*vargs, **kwargs):  
        primo_var_arg=vargs[0]  
        classe = type(primo_var_arg)  
        nuova_tupla = (classe,)  
        idx_varg = 1  
        while idx_varg < len(vargs):  
            nuova_tupla = nuova_tupla + (vargs[idx_varg],)  
            idx_varg = idx_varg + 1  
        funzione(*nuova_tupla, **kwargs)  
    return wrapper
```